



PROGRAMMATION ORIENTÉE OBJET II
INF11207 (MS)

UNIVERSITÉ DU QUÉBEC À RIMOUSKI
DÉPARTEMENT DE MATHÉMATIQUES, D'INFORMATIQUE ET DE GÉNIE

RAPPORT TP 2
Classification automatique des grains
de blé
Interface graphique MVVM

ÉTUDIANT(E)(S) :
RASANDIMANANA Tafita
ATTA Fidèle

PROFESSEUR :
Yacine Yaddaden, Ph. D.

Date : 10 mai 2026

Table des matières

1	Introduction	2
2	Analyse du problème et cahier des charges	3
2.1	Cahier des charges	3
3	Gestion de version avec Git et GitHub	3
3.1	Organisation du dépôt	3
3.2	Accès au code source	4
4	Présentation du code	4
4.1	Image	4
4.2	Migrations	4
4.2.1	20260507065730 InitialCreate	4
4.2.2	20260507065730 InitialCreate	5
4.2.3	ClassificationGrainDeBlesContextModelSnapshot	5
4.3	Models	7
4.3.1	ClassificationGrainDeBlesContext	7
4.3.2	ClassifierKnn	7
4.3.3	DataConverter	10
4.3.4	DistanceEuclidienne	12
4.3.5	DistanceManhattan	13
4.3.6	Donnee	13
4.3.7	Echantillon	14
4.3.8	EnsembleDonnees	14
4.3.9	EvaluationPerformance	15
4.3.10	Grain	18
4.3.11	IClassifier	19
4.3.12	IDistance	20
4.3.13	ShareModel	20
4.3.14	TypeDeGrain	21
4.3.15	Utilisateur	22
4.3.16	Voisin	22
4.4	Services	23
4.4.1	UtilisateurService	23
4.5	Views	24
4.5.1	MainWindow.xalm	24
4.5.2	MainWindow.xaml.cs	34
4.6	ViewModels	35
4.6.1	MainWindowViewModel.cs	35
4.6.2	RelayCommand.cs	49
4.7	Classe Converters.cs	50
4.8	App.xaml.cs	51
5	Conclusion	52

1 Introduction

Pour optimiser l'expérience utilisateur de l'application de classification des grains, ce second projet pratique implique la fourniture d'une interface graphique complète pour l'outil. La précédente application console se transforme alors en une application interactive capable de charger des fichiers de données, de configurer le classificateur k-NN et d'afficher les résultats de façon claire.

Afin de garantir la solidité et le suivi des classifications, les données issues des expériences sont archivées dans une base de données relationnelle à l'aide d'Entity Framework Core. De plus, une communication avec une API REST publique est intégrée pour enrichir l'application d'informations externes. Le projet entier est construit sur l'architecture MVVM (Modèle-Vue-ViewModel) et utilise les technologies Git, C# et WPF.

2 Analyse du problème et cahier des charges

2.1 Cahier des charges

Le projet répond aux exigences suivantes :

- Importation de données à partir d'un fichier CSV.
- Implémentation de l'algorithme KNN avec deux mesures de distance.
- Architecture orientée objet respectant les meilleures pratiques.
- Implémentation d'un système de persistance des données.
- Interface graphique pour la présentation des résultats.
- Gestion de version avec Git et GitHub.

3 Gestion de version avec Git et GitHub

Pour assurer un travail collaboratif efficace et sécurisé, nous avons mis en place un workflow Git structuré.

3.1 Organisation du dépôt

Le dépôt GitHub contient tout le code source, la documentation et les ressources nécessaires au projet. Nous avons utilisé le même dépôt git que le TP1 avec la création d'un dossier TP2.

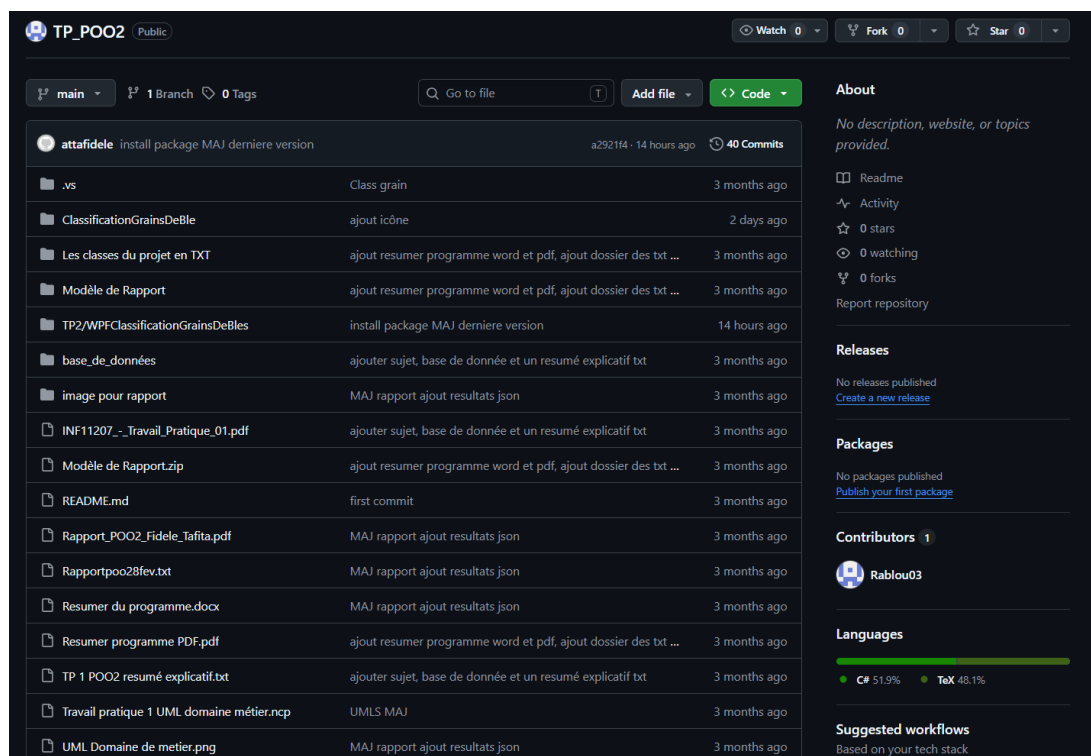


FIGURE 1 – Capture d'écran du dépôt GitHub montrant la structure du projet

3.2 Accès au code source

Le code source complet est disponible à l'adresse suivante :

https://github.com/Rablou03/TP_P002.git

4 Présentation du code

Nous présentons notre architecture logicielle MVVM (Model-View-ViewModel) pour la réalisation du projet, ainsi que le contenu des différents dossiers et de chaque classe.

4.1 Image

Le dossier contient toutes les images .ico utilisées pour les icônes.

4.2 Migrations

4.2.1 20260507065730 InitialCreate

Cette migration Entity Framework Core établit une table appelée "donnees" dans une base de données SQL Server.

Code de la migration :

```
1 using Microsoft.EntityFrameworkCore.Migrations;
2
3 namespace WPFClassificationGrainsDeBles.Migrations
4 {
5     public partial class InitialCreate : Migration
6     {
7         protected override void Up(MigrationBuilder
            migrationBuilder)
8         {
9             migrationBuilder.CreateTable(
10                 name: "donnees",
11                 columns: table => new
12                 {
13                     Id = table.Column<int>(nullable: false)
14                         .Annotation("SqlServer:Identity", "1, 1"),
15                     k = table.Column<int>(nullable: false),
16                     Distance = table.Column<double>(nullable: false
17                         ),
18                     donnee_Testster = table.Column<string>(nullable:
19                         true),
20                     precision = table.Column<string>(nullable: true
21                         )
22                 },
23                 constraints: table =>
24                 {
25                     table.PrimaryKey("PK_donnees", x => x.Id);
26                 }
27             }
28         }
29     }
```

```

25
26     protected override void Down(MigrationBuilder
27         migrationBuilder)
28     {
29         migrationBuilder.DropTable(
30             name: "donnees");
31     }
32 }

```

Listing 1 – Migration InitialCreate.cs

4.2.2 20260507065730 InitialCreate

Cette migration Entity Framework Core autorise l'ajout d'une nouvelle colonne à une table existante, tout en préservant les données actuelles.

```

1  using Microsoft.EntityFrameworkCore.Migrations;
2
3  namespace WPFClassificationGrainsDeBles.Migrations
4  {
5      public partial class AjoutAuteur : Migration
6      {
7          protected override void Up(MigrationBuilder
8              migrationBuilder)
9          {
10             migrationBuilder.AddColumn<string>(
11                 name: "AuteurNom",
12                 table: "donnees",
13                 nullable: true);
14
15             protected override void Down(MigrationBuilder
16                 migrationBuilder)
17             {
18                 migrationBuilder.DropColumn(
19                     name: "AuteurNom",
20                     table: "donnees");
21             }
22 }

```

Listing 2 – Migration AjoutAuteur.cs

4.2.3 ClassificationGrainDeBlesContextModelSnapshot

Code :

```

1  // <auto-generated />
2  using Microsoft.EntityFrameworkCore;
3  using Microsoft.EntityFrameworkCore.Infrastructure;
4  using Microsoft.EntityFrameworkCore.Metadata;
5  using Microsoft.EntityFrameworkCore.Storage.ValueConversion;

```

```

6  using WPFClassificationGrainsDeBles.Models;
7
8  namespace WPFClassificationGrainsDeBles.Migrations
9  {
10     [DbContext(typeof(ClassificationGrainDeBlesContext))]
11     partial class ClassificationGrainDeBlesContextModelSnapshot :
12         ModelSnapshot
13     {
14         protected override void BuildModel(ModelBuilder
15             modelBuilder)
16         {
17             #pragma warning disable 612, 618
18             modelBuilder
19                 .HasAnnotation("ProductVersion", "3.1.10")
20                 .HasAnnotation("Relational:MaxIdentifierLength",
21                     128)
22                 .HasAnnotation("SqlServer:ValueGenerationStrategy",
23                     SqlServerValueGenerationStrategy.IdentityColumn
24                     );
25
26             modelBuilder.Entity("WPFClassificationGrainsDeBles.
27                 Models.Donnee", b =>
28             {
29                 b.Property<int>("Id")
30                     .ValueGeneratedOnAdd()
31                     .HasColumnType("int")
32                     .HasAnnotation("SqlServer:
33                         ValueGenerationStrategy",
34                         SqlServerValueGenerationStrategy.
35                             IdentityColumn);
36
37                 b.Property<string>("AuteurNom")
38                     .HasColumnType("nvarchar(max)");
39
40                 b.Property<double>("Distance")
41                     .HasColumnType("float");
42
43                 b.Property<string>("donnee_Testeur")
44                     .HasColumnType("nvarchar(max)");
45
46                 b.Property<int>("k")
47                     .HasColumnType("int");
48
49                 b.Property<string>("precision")
50                     .HasColumnType("nvarchar(max)");
51
52                 b.HasKey("Id");
53
54                 b.ToTable("donnees");
55             });
56             #pragma warning restore 612, 618

```

```

50     }
51 }
52 }

```

Listing 3 – ClassificationGrainDeBlesContextModelSnapshot.cs

4.3 Models

Les classes situées dans le dossier `Models` représentent les entités professionnelles de l'application, comprenant la structure des données liées aux grains de blé ainsi que les résultats de leur classification. NB : Ces codes ont été vu au TP1. Nous expliquons les nouveaux comme le classe `ShareModel`.

4.3.1 ClassificationGrainDeBlesContext

```

1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6  using Microsoft.EntityFrameworkCore;
7
8  namespace WPFClassificationGrainsDeBles.Models
9  {
10     internal class ClassificationGrainDeBlesContext : DbContext
11     {
12         public DbSet<Models.Donnee> donnees { get; set; }
13
14         protected override void OnConfiguring(
15             DbContextOptionsBuilder dbContextOptionsBuilder)
16         {
17             string connection_string = "Data Source=(localdb)\\
18                                     MSSQLLocalDB;Initial Catalog=master;Integrated
19                                     Security=True;Connect Timeout=30;Encrypt=False;";
20             string database_name = "GrainsDB" +
21                                     "";
22             dbContextOptionsBuilder.UseSqlServer($"{
23                                     connection_string}; " +
24                                     $"Database ={database_name};");
25         }
26     }
27 }

```

Listing 4 – Contexte de base de données ClassificationGrainDeBlesContext.cs

4.3.2 ClassifierKnn

```

1  using System;
2  using System.Collections.Generic;
3  using System.Linq;

```



```

4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace WPFClassificationGrainsDeBles.Models
8  {
9      public class ClassifierKnn : IClassifier
10     {
11         IDistance distance;
12         EnsembleDonnees training;
13         int k;
14
15         public ClassifierKnn(int k, IDistance distance)
16         {
17             this.k = k;
18             this.distance = distance;
19         }
20
21         public void Entrainer(EnsembleDonnees donnees)
22         {
23             training = donnees;
24         }
25
26         public List<Voisin> ListeTousVoisin(double[]
           caracteristiques)
27         {
28             List<Voisin> liste = new List<Voisin>();
29
30             foreach (Echantillon e in training.ObtenirEchantillon()
31                 )
32             {
33                 double d = distance.Calculer(caracteristiques, e.
34                     Caracteristiques);
35                 liste.Add(new Voisin(e, d));
36             }
37
38             return liste;
39
40         }
41
42         public List<Voisin> TrierListVoisinParDistance(List<Voisin>
           voisins)
43         {
44             if (voisins.Count <= 1)
45                 return voisins;
46
47             Voisin pivot = voisins[0];
48
49             List<Voisin> plusPetits = new List<Voisin>();
50             List<Voisin> plusGrands = new List<Voisin>();
51
52             for (int i = 1; i < voisins.Count; i++)
53             {

```

```

51         if (voisins[i].Distance < pivot.Distance)
52             plusPetits.Add(voisins[i]);
53         else
54             plusGrands.Add(voisins[i]);
55     }
56
57     List<Voisin> resultat = new List<Voisin>();
58
59     resultat.AddRange(TrierListVoisinParDistance(plusPetits
60         ));
61     resultat.Add(pivot);
62     resultat.AddRange(TrierListVoisinParDistance(plusGrands
63         ));
64
65     return resultat;
66 }
67
68 public TypeDeGrain VoteMajoritaire(List<Voisin> voisins)
69 {
70     TypeDeGrain typeDuGrain = new TypeDeGrain();
71     int compteurKama = 0;
72     int compteurRosa = 0;
73     int compteurCanadian = 0;
74
75     foreach (var voisin in voisins)
76     {
77         if (voisin.Echantillon.Etiquette.ToString() == "
78             Kama")
79         {
80             compteurKama++;
81         }
82         if (voisin.Echantillon.Etiquette.ToString() == "
83             Rosa")
84         {
85             compteurRosa++;
86         }
87         if (voisin.Echantillon.Etiquette.ToString() == "
88             Canadian")
89         {
90             compteurCanadian++;
91         }
92     }
93     if (compteurKama >= compteurRosa && compteurKama >=
94         compteurCanadian)
95         typeDuGrain = TypeDeGrain.Kama;
96
97     else if (compteurRosa >= compteurKama && compteurRosa
98         >= compteurCanadian)
99         typeDuGrain = TypeDeGrain.Rosa;
100
101     else

```

```

95         typeDuGrain = TypeDeGrain.Canadian;
96
97         return typeDuGrain;
98
99     }
100
101     public TypeDeGrain Predire(Echantillon e)
102     {
103         List<Voisin> voisins = ListeTousVoisin(e.
            Caracteristiques);
104         List<Voisin> voisinsTries = TrierListVoisinParDistance(
            voisins);
105
106         List<Voisin> kVoisins = new List<Voisin>();
107         for (int i = 0; i < k; i++)
108         {
109             kVoisins.Add(voisinsTries[i]);
110         }
111
112         return VoteMajoritaire(kVoisins);
113     }
114 }
115

```

Listing 5 – Classe ClassifierKnn.cs

4.3.3 DataConverter

```

1  using System;
2  using System.Collections.Generic;
3  using System.Data;
4  using System.Globalization;
5  using System.Linq;
6  using System.Text;
7  using System.Threading.Tasks;
8  using System.Windows.Controls;
9  using System.Windows.Documents;
10 using System.IO;
11
12 namespace WPFClassificationGrainsDeBles.Models
13 {
14     public class DataConverter // Chang de Convert
        DataConverter
15     {
16         public static List<Grain> ConversionListe(string
            nom_fichier)
17         {
18             List<Grain> grains = new List<Grain>();
19
20             if (!File.Exists(nom_fichier))
21             {

```

```

22         throw new FileNotFoundException($"Fichier non
23         trouv : {nom_fichier}");
24     }
25
26     string[] lignes = File.ReadAllLines(nom_fichier);
27     if (lignes.Length == 0) return grains;
28
29     string[] headers = lignes[0].Split(';');
30
31     int idxVariety = Array.IndexOf(headers, "variety");
32     int idxArea = Array.IndexOf(headers, "Area");
33     int idxPerimeter = Array.IndexOf(headers, "Perimeter");
34     int idxCompactness = Array.IndexOf(headers, "
35     Compactness");
36     int idxKernelLength = Array.IndexOf(headers, "
37     Kernel_Length");
38     int idxKernelWidth = Array.IndexOf(headers, "
39     Kernel_Width");
40     int idxAsymmetry = Array.IndexOf(headers, "
41     Asymmetry_Coefficient");
42     int idxGroove = Array.IndexOf(headers, "Groove_Length")
43     ;
44
45     for (int i = 1; i < lignes.Length; i++)
46     {
47         string[] colonnes = lignes[i].Split(';');
48         if (colonnes.Length < 8) continue;
49
50         try
51         {
52             var g = new Grain(
53                 (TypeDeGrain)Enum.Parse(typeof(TypeDeGrain)
54                 , colonnes[idxVariety]),
55                 double.Parse(colonnes[idxArea], CultureInfo
56                 .InvariantCulture),
57                 double.Parse(colonnes[idxPerimeter],
58                 CultureInfo.InvariantCulture),
59                 double.Parse(colonnes[idxCompactness],
60                 CultureInfo.InvariantCulture),
61                 double.Parse(colonnes[idxKernelLength],
62                 CultureInfo.InvariantCulture),
63                 double.Parse(colonnes[idxKernelWidth],
64                 CultureInfo.InvariantCulture),
65                 double.Parse(colonnes[idxAsymmetry],
66                 CultureInfo.InvariantCulture),
67                 double.Parse(colonnes[idxGroove],
68                 CultureInfo.InvariantCulture)
69             );
70             grains.Add(g);
71         }
72         catch (Exception ex)

```

```

59         {
60             Console.WriteLine($"Erreur ligne {i}: {ex.
                                   Message}");
61         }
62     }
63
64     return grains;
65 }
66
67 public static void SaveEchantillon(List<Grain> grains,
68     EnsembleDonnees ensemble)
69 {
70     if (grains == null || ensemble == null) return;
71
72     foreach (var g in grains)
73     {
74         double[] carac = new double[]
75         {
76             g.Area, g.Perimeter, g.Compactness,
77             g.LongueurNoyau, g.LargeurNoyau,
78             g.AsymetryCoefficient, g.GrooveLength
79         };
80
81         ensemble.AjouterUnEchantillon(new Echantillon(carac
82             , g.TypeDeGrain));
83     }
84 }

```

Listing 6 – Classe DataConverter.cs

4.3.4 DistanceEuclidienne

```

1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace WPFClassificationGrainsDeBles.Models
8  {
9      public class DistanceEuclidienne : IDistance
10     {
11         public double Calculer(double[] a, double[] b)
12         {
13             double somme = 0;
14             for (int i = 0; i < a.Length; i++)
15             {
16                 somme += Math.Pow(a[i] - b[i], 2);
17             }
18         }
19     }
20 }

```

```

18         return Math.Sqrt(somme);
19     }
20 }
21 }

```

Listing 7 – Classe DistanceEuclidienne.cs

4.3.5 DistanceManhattan

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace WPFClassificationGrainsDeBles.Models
8 {
9     public class DistanceManhattan : IDistance
10    {
11        public double Calculer(double[] a, double[] b)
12        {
13            double somme = 0;
14            for (int i = 0; i < a.Length; i++)
15            {
16                somme += Math.Abs(a[i] - b[i]);
17            }
18            return somme;
19        }
20    }
21 }

```

Listing 8 – Classe DistanceManhattan.cs

4.3.6 Donnee

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace WPFClassificationGrainsDeBles.Models
8 {
9     internal class Donnee
10    {
11
12        public int Id { get; set; }
13
14        public int k { get; set; }
15        public double Distance { get; set; }
16    }
17 }

```

```

16         public string donnee_Tester { get; set; }
17
18         public string precision { get; set; }
19
20         public string AuteurNom { get; set; }
21
22     }
23 }

```

Listing 9 – Classe Donnee.cs

4.3.7 Echantillon

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace WPFClassificationGrainsDeBles.Models
8 {
9     public class Echantillon
10    {
11        public double[] Caracteristiques { get; set; }
12        public TypeDeGrain Etiquette { get; set; }
13
14        public Echantillon(double[] caracteristiques, TypeDeGrain
            etiquette)
15        {
16            Caracteristiques = caracteristiques;
17            Etiquette = etiquette;
18        }
19    }
20 }

```

Listing 10 – Classe Echantillon.cs

4.3.8 EnsembleDonnees

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace WPFClassificationGrainsDeBles.Models
8 {
9     public class EnsembleDonnees
10    {
11        private List<Echantillon> echantillons = new List<
            Echantillon>();

```

```

12     public void AjouterUnEchantillon(Echantillon e)
13     {
14         echantillons.Add(e);
15     }
16
17     public void AjouterListEchantillon(List<Echantillon>
18         echantillons)
19     {
20         foreach (Echantillon e in echantillons)
21         {
22             echantillons.Add(e);
23         }
24     }
25     public List<Echantillon> ObtenirEchantillon()
26     {
27         return echantillons;
28     }
29     public int Taille()
30     {
31         return echantillons.Count;
32     }
33 }

```

Listing 11 – Classe EnsembleDonnees.cs

4.3.9 EvaluationPerformance

```

1  using Newtonsoft.Json;
2  using System;
3  using System.Collections.Generic;
4  using System.Data;
5  using System.Linq;
6  using System.Text;
7  using System.Threading.Tasks;
8  using System.Windows.Documents;
9  using System.IO;
10 using System.Text.Json;
11 using JsonSerializer = System.Text.Json.JsonSerializer;
12
13 namespace WPFClassificationGrainsDeBles.Models
14 {
15     public class EvaluationPerformance
16     {
17         private readonly int[,] matrice;
18         private int total;
19         private int correct;
20
21         public EvaluationPerformance()
22         {
23             matrice = new int[3, 3];

```



```

24         total = 0;
25         correct = 0;
26     }
27
28     public void Evaluer(int k, IDistance distance,
29         EnsembleDonnees train, EnsembleDonnees test)
30     {
31         ClassifierKnn classifier = new ClassifierKnn(k,
32             distance);
33         classifier.Entrainer(train);
34
35         foreach (var e in test.ObtenirEchantillon())
36         {
37             TypeDeGrain reel = e.Etiquette;
38             TypeDeGrain predition = classifier.Predire(e);
39
40             if (reel == predition)
41                 correct++;
42
43             int ligne = GetIndex(reel);
44             int colonne = GetIndex(predition);
45
46             matrice[ligne, colonne]++;
47             total++;
48         }
49     }
50
51     public int GetIndex(TypeDeGrain type)
52     {
53         return type switch
54         {
55             TypeDeGrain.Kama => 0,
56             TypeDeGrain.Rosa => 1,
57             _ => 2 // Canadian
58         };
59     }
60
61     public double CalculerAccuracy()
62     {
63         if (total == 0) return 0;
64         return (double)correct / total;
65     }
66
67     // Version WPF : retourne un DataTable pour affichage dans
68     // un DataGridView
69     public DataTable ConstruireTableauWpf()
70     {
71         DataTable table = new DataTable();
72
73         table.Columns.Add("R e l \\ Pr d i t");
74         table.Columns.Add("Kama");
75     }

```

```

72     table.Columns.Add("Rosa");
73     table.Columns.Add("Canadian");
74
75     string[] labels = { "Kama", "Rosa", "Canadian" };
76
77     for (int i = 0; i < 3; i++)
78     {
79         table.Rows.Add(
80             labels[i],
81             matrice[i, 0],
82             matrice[i, 1],
83             matrice[i, 2]
84         );
85     }
86
87     return table;
88 }
89
90 public void SauvegarderJsonGlobal(string chemin, int k,
91     IDistance typeDistance, EnsembleDonnees train,
92     EnsembleDonnees test)
93 {
94     var matriceListe = new List<List<int>>>();
95     for (int i = 0; i < 3; i++)
96     {
97         var ligne = new List<int>();
98         for (int j = 0; j < 3; j++)
99             ligne.Add(matrice[i, j]);
100
101         matriceListe.Add(ligne);
102     }
103
104     var nouveauResultat = new
105     {
106         ParametresExecution = new
107         {
108             k = k,
109             distance = typeDistance.ToString(),
110             date = DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"),
111         },
112         JeuxDeDonnees = new
113         {
114             taille_train = train.Taille(),
115             taille_test = test.Taille()
116         },
117         Evaluation = new
118         {
119             accuracy = CalculerAccuracy(),
120             matrice_confusion = matriceListe
121         }
122     }

```

```

120         };
121
122         List<object> listeResultats;
123
124         if (File.Exists(chemin))
125         {
126             string ancienJson = File.ReadAllText(chemin);
127             try
128             {
129                 listeResultats = JsonSerializer.Deserialize<
130                     List<object>>(ancienJson) ?? new List<object>
131                     >();
132             }
133             catch
134             {
135                 listeResultats = new List<object>();
136             }
137         }
138         else
139         {
140             listeResultats = new List<object>();
141         }
142
143         listeResultats.Add(nouveauResultat);
144
145         string jsonFinal = JsonSerializer.Serialize(
146             listeResultats,
147             new JsonSerializerOptions { WriteIndented = true })
148             ;
149
150         File.WriteAllText(chemin, jsonFinal);
151
152         Console.WriteLine($"Fichier JSON mis à jour ici : {
153             Path.GetFullPath(chemin)}");
154     }
155 }
156 }

```

Listing 12 – Classe EvaluationPerformance.cs

4.3.10 Grain

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace WPFClassificationGrainsDeBles.Models
8 {
9     public class Grain

```

```

10 {
11     TypeDeGrain typeDeGrain;
12     double area;
13     double perimeter;
14     double compactness;
15     double longueurNoyau;
16     double largeurNoyau;
17     double asymetryCoefficient;
18     double grooveLength;
19
20     public Grain(TypeDeGrain typeDeGrain, double area, double
        perimeter, double compactness, double longueurNoyau,
        double largeurNoyau, double asymetryCoefficient, double
        grooveLength)
21     {
22         this.typeDeGrain = typeDeGrain;
23         this.area = area;
24         this.perimeter = perimeter;
25         this.compactness = compactness;
26         this.longueurNoyau = longueurNoyau;
27         this.largeurNoyau = largeurNoyau;
28         this.asymetryCoefficient = asymetryCoefficient;
29         this.grooveLength = grooveLength;
30     }
31
32     public TypeDeGrain TypeDeGrain { get { return typeDeGrain;
        } set { typeDeGrain = value; } }
33     public double Area { get { return area; } set { area =
        value; } }
34     public double Perimeter { get { return perimeter; } set {
        perimeter = value; } }
35     public double Compactness { get { return compactness; } set
        { compactness = value; } }
36     public double LongueurNoyau { get { return longueurNoyau; }
        set { longueurNoyau = value; } }
37     public double LargeurNoyau { get { return largeurNoyau; }
        set { largeurNoyau = value; } }
38     public double AsymetryCoefficient { get { return
        asymetryCoefficient; } set { asymetryCoefficient = value
        ; } }
39     public double GrooveLength { get { return grooveLength; }
        set { grooveLength = value; } }
40
41 }
42 }

```

Listing 13 – Classe Grain.cs

4.3.11 IClassifier

```

1 using System;

```

```

2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace WPFClassificationGrainsDeBles.Models
8 {
9     public interface IClassifier
10    {
11        void Entrainer(EnsembleDonnees data);
12        TypeDeGrain Predire(Echantillon e);
13    }
14 }

```

Listing 14 – Interface IClassifier.cs

4.3.12 IDistance

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace WPFClassificationGrainsDeBles.Models
8 {
9     public interface IClassifier
10    {
11        void Entrainer(EnsembleDonnees data);
12        TypeDeGrain Predire(Echantillon e);
13    }
14 }

```

Listing 15 – Interface IClassifier.cs

4.3.13 ShareModel

Dans l'architecture MVVM, cette classe SharedModel sert de modèle partagé. Elle regroupe les informations et l'état de l'application afin de les rendre disponibles pour les différents ViewModels.

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace WPFClassificationGrainsDeBles.Models
8 {
9     public class SharedModel
10    {

```

```

11     public EnsembleDonnees TrainingData { get; set; } = new
        EnsembleDonnees();
12     public EnsembleDonnees TestData { get; set; } = new
        EnsembleDonnees();
13     public int K { get; set; } = 3;
14     public IDistance DistanceStrategy { get; set; }
15     public ClassifierKnn Classifier { get; set; }
16     public bool IsDataLoaded { get; set; } = false;
17
18     public SharedModel()
19     {
20         DistanceStrategy = new DistanceEuclidienne();
21     }
22
23     public void SetDistance(string distanceName)
24     {
25         if (distanceName == "Manhattan")
26             DistanceStrategy = new DistanceManhattan();
27         else
28             DistanceStrategy = new DistanceEuclidienne();
29     }
30
31     public string GetDistanceName()
32     {
33         return DistanceStrategy is DistanceManhattan ? "
            Manhattan" : "Euclidienne";
34     }
35
36     public void InitializeClassifier()
37     {
38         if (TrainingData != null && TrainingData.Taille() > 0)
39         {
40             Classifier = new ClassifierKnn(K, DistanceStrategy)
41             ;
42             Classifier.Entrainer(TrainingData);
43         }
44     }
45 }

```

Listing 16 – Classe SharedModel.cs

4.3.14 TypeDeGrain

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace WPFClassificationGrainsDeBles.Models

```

```

8 {
9     public enum TypeDeGrain
10    {
11        Kama ,
12        Rosa ,
13        Canadian ,
14    }
15 }

```

Listing 17 – Enumération TypeDeGrain.cs

4.3.15 Utilisateur

```

1 using System.Collections.Generic;
2 using System.Text.Json.Serialization;
3
4 namespace WPFClassificationGrainsDeBles.Models
5 {
6     public class Utilisateur
7     {
8         [JsonPropertyName("id")]
9         public int Id { get; set; }
10
11         [JsonPropertyName("firstName")]
12         public string FirstName { get; set; }
13
14         [JsonPropertyName("lastName")]
15         public string LastName { get; set; }
16
17         [JsonPropertyName("email")]
18         public string Email { get; set; }
19
20
21         public string NomComplet => $"{FirstName} {LastName}";
22     }
23
24     public class ReponseUtilisateurs
25     {
26         [JsonPropertyName("users")]
27         public List<Utilisateur> Users { get; set; }
28     }
29 }

```

Listing 18 – Classe Utilisateur et ReponseUtilisateurs.cs

4.3.16 Voisin

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;

```

```

4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace WPFClassificationGrainsDeBles.Models
8 {
9     public class Voisin
10    {
11        public Echantillon Echantillon { get; set; }
12        public double Distance { get; set; }
13
14        public Voisin(Echantillon e, double d)
15        {
16            Echantillon = e;
17            Distance = d;
18        }
19    }
20 }

```

Listing 19 – Classe Voisin.cs

4.4 Services

Le dossier **Services** contient les classes responsables de la logique métier : implémentation de l'algorithme k-NN, calcul des distances, persistance des données et communication avec l'API REST.

4.4.1 UtilisateurService

Cette classe **UtilisateurService** est un service qui permet de récupérer des utilisateurs depuis une API REST externe <https://dummyjson.com/>.

```

1 using System;
2 using System.Collections.Generic;
3 using System.Net.Http;
4 using System.Text.Json;
5 using System.Threading.Tasks;
6 using WPFClassificationGrainsDeBles.Models;
7
8 namespace WPFClassificationGrainsDeBles.Services
9 {
10    public class UtilisateurService
11    {
12        private readonly HttpClient _httpClient = new HttpClient();
13        private const string URL = "https://dummyjson.com/users";
14
15        public async Task<List<Utilisateur>> GetUtilisateursAsync()
16        {
17            try
18            {
19                string reponse = await _httpClient.GetStringAsync(
20                    URL);

```



```

21         var options = new JsonSerializerOptions
22         {
23             PropertyNameCaseInsensitive = true
24         };
25
26         var resultat = JsonSerializer.Deserialize<
27             ReponseUtilisateurs>(reponse, options);
28         return resultat?.Users ?? new List<Utilisateur>();
29     }
30     catch (Exception ex)
31     {
32         throw new Exception($"Erreur lors de l'appel API :
33             {ex.Message}");
34     }
35 }

```

Listing 20 – Service UtilisateurService.cs

4.5 Views

Le dossier Views regroupe les fichiers XAML définissant l'interface utilisateur de l'application WPF.

4.5.1 MainWindow.xalm

Le fichier zaml est le fichier qui définit l'interface graphique de la fenêtre principale de l'application.

```

1 <Window x:Class="WPFClassificationGrainsDeBles.View.MainWindow"
2     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/
3     presentation"
4     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
5     xmlns:viewmodels="clr-namespace:
6         WPFClassificationGrainsDeBles.ViewModels"
7     Title="Classification des grains de bl - k-NN"
8     Height="800"
9     Width="1400"
10    WindowStartupLocation="CenterScreen">
11
12    <Window.DataContext>
13        <viewmodels:MainWindowViewModel/>
14    </Window.DataContext>
15
16    <Grid>
17
18        <Grid.RowDefinitions>
19            <RowDefinition Height="Auto"/>
20            <RowDefinition Height="*/>

```

```

21 <!-- MENU HORIZONTAL EN HAUT -->
22 <Border BorderBrush="#CCCCCC"
23     BorderThickness="0,0,0,1"
24     Background="#F5F5F5">
25
26     <ScrollView HorizontalScrollBarVisibility="Auto"
27         VerticalScrollBarVisibility="Disabled">
28
29         <StackPanel Orientation="Horizontal" Margin="10,5">
30
31             <!-- ACCUEIL -->
32             <Button Command="{Binding NavigateHomeCommand}"
33                 Height="40"
34                 Width="130"
35                 Margin="5,0"
36                 ToolTip="Accueil">
37                 <StackPanel Orientation="Horizontal"
38                     HorizontalAlignment="Center">
39                     <Image Source="/Image/home-button_icons.com_72700.ico"
40                         Width="20" Height="20" Margin="
41                             0,0,5,0"/>
42                     <TextBlock Text="Accueil"
43                         VerticalAlignment="Center"/>
44                 </StackPanel>
45             </Button>
46
47             <Separator Style="{StaticResource {x:Static
48                 Toolbar.SeparatorStyleKey}}"
49                 Width="1" Background="Gray" Margin="
50                     5,0"/>
51
52             <!-- IMPORTER -->
53             <Button Command="{Binding NavigateImportCommand
54                 }"
55                 Height="40" Width="130" Margin="5,0"
56                 ToolTip="Importer les donn es">
57                 <StackPanel Orientation="Horizontal"
58                     HorizontalAlignment="Center">
59                     <Image Source="/Image/import_120260.ico"
60                         Width="20" Height="20" Margin="
61                             0,0,5,0"/>
62                     <TextBlock Text="Importer"
63                         VerticalAlignment="Center"/>
64                 </StackPanel>
65             </Button>
66
67             <!-- CONFIGURER K -->
68             <Button Command="{Binding
69                 NavigateKConfigCommand}"

```

```

60         Height="40" Width="130" Margin="5,0"
61         ToolTip="Choisir la valeur de k">
62     <StackPanel Orientation="Horizontal"
63         HorizontalAlignment="Center">
64         <Image Source="/Image/3844439-gear-
65             setting-settings-wheel_110294.ico"
66             Width="20" Height="20" Margin="
67                 0,0,5,0"/>
68         <TextBlock Text="Configurer k"
69             VerticalAlignment="Center"/>
70     </StackPanel>
71 </Button>
72
73 <!-- DISTANCE -->
74 <Button Command="{Binding
75     NavigateDistanceConfigCommand}"
76     Height="40" Width="130" Margin="5,0"
77     ToolTip="Choisir la distance">
78     <StackPanel Orientation="Horizontal"
79         HorizontalAlignment="Center">
80         <Image Source="/Image/
81             calculate_equals_icon_194844.ico"
82             Width="20" Height="20" Margin="
83                 0,0,5,0"/>
84         <TextBlock Text="Distance"
85             VerticalAlignment="Center"/>
86     </StackPanel>
87 </Button>
88
89 <Separator Style="{StaticResource {x:Static
90     Toolbar.SeparatorStyleKey}}"
91     Width="1" Background="Gray" Margin="
92         5,0"/>
93
94 <!-- ENTRAÎNER -->
95 <Button Command="{Binding TrainCommand}"
96     Height="40" Width="130" Margin="5,0"
97     ToolTip="Entrainer le modèle">
98     <StackPanel Orientation="Horizontal"
99         HorizontalAlignment="Center">
100         <Image Source="/Image/run-icon-icons.
101             com_61189.ico"
102             Width="20" Height="20" Margin="
103                 0,0,5,0"/>
104         <TextBlock Text="Entrainer"
105             VerticalAlignment="Center"/>
106     </StackPanel>
107 </Button>
108
109 <!-- TESTER -->
110 <Button Command="{Binding TestCommand}"

```

```

96         Height="40" Width="130" Margin="5,0"
97         Tooltip="Tester le module">
98     <StackPanel Orientation="Horizontal"
99         HorizontalAlignment="Center">
100         <Image Source="/Image/
101             PixelKit_0034_Waiting-for-
102             Validation_37731.ico"
103             Width="20" Height="20" Margin="
104                 0,0,5,0"/>
105         <TextBlock Text="Tester"
106             VerticalAlignment="Center"/>
107     </StackPanel>
108 </Button>
109
110 <Separator Style="{StaticResource {x:Static
111     ToolBar.SeparatorStyleKey}}"
112     Width="1" Background="Gray" Margin="
113         5,0"/>
114
115 <!-- RESULTATS -->
116 <Button Command="{Binding
117     NavigateResultsCommand}"
118     Height="40" Width="130" Margin="5,0"
119     Tooltip="Voir les resultats">
120     <StackPanel Orientation="Horizontal"
121         HorizontalAlignment="Center">
122         <Image Source="/Image/findings_icon-
123             icons.com_67772.ico"
124             Width="20" Height="20" Margin="
125                 0,0,5,0"/>
126         <TextBlock Text="Resultats"
127             VerticalAlignment="Center"/>
128     </StackPanel>
129 </Button>

```

```

130 <!-- HISTORIQUE -->
131 <Button Command="{Binding
132     NavigateHistoryCommand}"
133     Height="40" Width="130" Margin="5,0"
134     Tooltip="Voir l'historique">
135     <StackPanel Orientation="Horizontal"
136         HorizontalAlignment="Center">
137         <Image Source="/Image/archive_120061.
138             ico"
139             Width="20" Height="20" Margin="
140                 0,0,5,0"/>
141         <TextBlock Text="Historique"
142             VerticalAlignment="Center"/>
143     </StackPanel>
144 </Button>

```

```

130         <Separator Style="{StaticResource {x:Static
131             ToolBar.SeparatorStyleKey}}"
132             Width="1" Background="Gray" Margin="
133                 5,0"/>
134
135     <!-- QUITTER -->
136     <Button Command="{Binding QuitCommand}"
137         Height="40" Width="130" Margin="5,0"
138         ToolTip="Quitter l'application">
139         <StackPanel Orientation="Horizontal"
140             HorizontalAlignment="Center">
141             <Image Source="/Image/4213459-common-
142                 door-exit-logout-out-signout_115411.
143                 ico"
144                 Width="20" Height="20" Margin="
145                     0,0,5,0"/>
146             <TextBlock Text="Quitter"
147                 VerticalAlignment="Center"/>
148         </StackPanel>
149     </Button>
150
151     </StackPanel>
152     </ScrollViewer>
153 </Border>
154
155 <!-- CONTENU PRINCIPAL -->
156 <ScrollViewer Grid.Row="1" VerticalScrollBarVisibility="
157     Auto">
158
159     <StackPanel Margin="15">
160
161         <!-- EN-TETE -->
162         <Border Background="#E3F2FD" Padding="10"
163             CornerRadius="5" Margin="0,0,0,15">
164             <TextBlock Text="{Binding CurrentTitle}"
165                 FontSize="20" FontWeight="Bold"
166                 HorizontalAlignment="Center"
167                 Foreground="#1565C0"/>
168         </Border>
169
170         <!-- PAGE D'ACCUEIL -->
171         <Border Background="#E8F5E9" Padding="30"
172             CornerRadius="10"
173             Margin="0,0,0,15" Visibility="{Binding
174                 ShowWelcome}">
175             <StackPanel HorizontalAlignment="Center">
176                 <TextBlock Text="Classification des grains
177                     de bl avec k-NN"
178                     FontSize="18"
179                     HorizontalAlignment="Center"
180                     Margin="0,10,0,0" Foreground

```

```

168        ="#1565C0"/>
169     <TextBlock Text="Utilisez le menu ci-dessus
170         pour naviguer dans l'application"
171         FontSize="14"
172         HorizontalAlignment="Center"
173         Margin="0,20,0,0" Foreground="
174         Gray"/>
175     </StackPanel>
176 </Border>
177
178 <!-- IMPORTATION -->
179 <GroupBox Header="1. Importer les donn es "
180     Visibility="{Binding ShowImport}">
181     <StackPanel Margin="10">
182         <TextBlock Text="Fichier d'entra nement (
183             train.csv):" FontWeight="Bold"/>
184         <Grid Margin="0,5,0,10">
185             <Grid.ColumnDefinitions>
186                 <ColumnDefinition Width="*/>
187                 <ColumnDefinition Width="Auto"/>
188             </Grid.ColumnDefinitions>
189             <TextBox Text="{Binding TrainPath}"
190                 IsReadOnly="True" Background="#
191                 F9F9F9"/>
192             <Button Content="Parcourir" Command="{
193                 Binding BrowseTrainCommand}"
194                 Grid.Column="1" Margin
195                 ="5,0,0,0" Width="80" Height
196                 ="30"/>
197         </Grid>
198
199         <TextBlock Text="Fichier de test (test.csv)
200             : " FontWeight="Bold" Margin="0,10,0,0"/>
201         <Grid Margin="0,5,0,10">
202             <Grid.ColumnDefinitions>
203                 <ColumnDefinition Width="*/>
204                 <ColumnDefinition Width="Auto"/>
205             </Grid.ColumnDefinitions>
206             <TextBox Text="{Binding TestPath}"
207                 IsReadOnly="True" Background="#
208                 F9F9F9"/>
209             <Button Content="Parcourir" Command="{
210                 Binding BrowseTestCommand}"
211                 Grid.Column="1" Margin
212                 ="5,0,0,0" Width="80" Height
213                 ="30"/>
214         </Grid>
215
216         <Button Content="Importer les donn es "
217             Command="{Binding ImportCommand}"

```

```

202         Height="40" Margin="0,10"
203         FontWeight="Bold"
204         Background="#4CAF50" Foreground="
205         White"/>
206
207     <TextBlock Text="Statut:" FontWeight="Bold"
208         Margin="0,10,0,0"/>
209     <Border Background="#F0F0F0" Padding="5"
210         CornerRadius="3">
211         <TextBlock Text="{Binding ImportStatus
212             }" TextWrapping="Wrap" FontSize
213             ="11"/>
214     </Border>
215
216 </StackPanel>
217 </GroupBox>
218
219 <!-- CONFIGURATION K -->
220 <GroupBox Header="2. Choisir la valeur de k"
221     Visibility="{Binding ShowKConfig}" Margin
222     ="0,15,0,0">
223     <StackPanel Margin="10">
224
225         <TextBlock Text="Veuillez configurer k
226             avant de choisir la distance"
227             FontWeight="Bold" Foreground
228             ="#1565C0"
229             HorizontalAlignment="Center"
230             Margin="0,0,0,15"/>
231
232         <StackPanel Orientation="Horizontal"
233             HorizontalAlignment="Center">
234             <TextBlock Text="k = " FontSize="20"
235                 VerticalAlignment="Center"
236                 FontWeight="Bold"/>
237             <TextBox Text="{Binding KValue,
238                 UpdateSourceTrigger=PropertyChanged
239                 }"
240                 Width="80" Height="40"
241                 TextAlignment="Center"
242                 FontSize="16" Margin="10,0"/>
243         </StackPanel>
244
245         <TextBlock Text="(valeur entre 1 et 30)"
246             FontSize="12"
247             Foreground="Gray"
248             HorizontalAlignment="Center"
249             Margin="0,5"/>
250
251         <Button Content="Valider k et continuer"
252             Command="{Binding

```

```

234         ValidateKAndContinueCommand}"
235         Height="40" Width="200" Margin
236         ="0,15"
237         HorizontalAlignment="Center"
238         Background="#2196F3" Foreground="
239         White" FontWeight="Bold"/>
240
241     </StackPanel>
242 </GroupBox>
243
244 <!-- DISTANCE -->
245 <GroupBox Header="3. Choisir la distance"
246     Visibility="{Binding ShowDistanceConfig}"
247     Margin="0,15,0,0">
248     <StackPanel Margin="10">
249
250         <TextBlock Text="Choisissez la distance
251             pour la classification"
252             FontWeight="Bold" Foreground
253             ="#1565C0"
254             HorizontalAlignment="Center"
255             Margin="0,0,0,15"/>
256
257         <RadioButton Content="Euclidienne"
258             IsChecked="{Binding IsEuclidienne}"
259             Margin="0,10" FontSize="14"
260             GroupName="Distance"/>
261         <RadioButton Content="Manhattan" IsChecked
262             ="{Binding IsManhattan}"
263             Margin="0,10" FontSize="14"
264             GroupName="Distance"/>
265
266         <Button Content="Valider la distance et
267             continuer"
268             Command="{Binding
269                 ValidateDistanceAndContinueCommand
270             }"
271             Height="40" Width="250" Margin
272             ="0,20,0,0"
273             HorizontalAlignment="Center"
274             Background="#FF9800" Foreground="
275             White" FontWeight="Bold"/>
276
277     </StackPanel>
278 </GroupBox>
279
280 <!-- ENTRAINEMENT -->
281 <GroupBox Header="4. Entra ner le mod le"
282     Visibility="{Binding ShowTrain}" Margin
283     ="0,15,0,0">
284     <StackPanel Margin="10">

```



```

268
269         <TextBlock Text="Configuration actuelle :"
270             FontWeight="Bold" Margin="0,0,0,10"/>
271         <Border Background="#F0F0F0" Padding="10"
272             CornerRadius="5" Margin="0,0,0,15">
273             <TextBlock Text="{Binding
274                 CurrentConfigText}"
275                 FontFamily="Consolas"
276                 FontSize="12"/>
277         </Border>
278
279         <Button Content="Entra ner le mod le "
280             Command="{Binding TrainCommand}"
281             Height="45" Margin="0,5"
282             Background="#2196F3" Foreground="
283                 White"
284             FontWeight="Bold" FontSize="14"/>
285
286     </StackPanel>
287 </GroupBox>
288
289 <!-- TEST -->
290 <GroupBox Header="5. Tester le mod le "
291     Visibility="{Binding ShowTest}" Margin
292     ="0,15,0,0">
293     <StackPanel Margin="10">
294
295         <!-- SECTION AUTEUR -->
296         <Border Background="#FFF8E1" Padding="10"
297             CornerRadius="5" Margin="0,0,0,15"
298             BorderBrush="#FFB300"
299             BorderThickness="1">
300             <StackPanel>
301
302                 <TextBlock Text="Choisir un auteur
303                     pour cette exp rience "
304                     FontWeight="Bold"
305                     Foreground="#E65100"
306                     Margin="0,0,0,10"/>
307
308                 <Button Content="Charger les
309                     utilisateurs depuis l'API"
310                     Command="{Binding
311                         ChargerUtilisateursCommand
312                     }"
313                     Height="35" Width="260"
314                     HorizontalAlignment="Left"
315                     Background="#FF9800"
316                     Foreground="White"
317                     FontWeight="Bold" Margin="
318                         0,0,0,10"/>

```

```

304
305         <TextBlock Text="S lectionner un
306             auteur : "
307             FontWeight="Bold" Margin
308                 ="0,0,0,5"/>
309
310         <ComboBox ItemsSource="{Binding
311             Utilisateurs}"
312             SelectedItem="{Binding
313                 UtilisateurSelectionne
314                 }"
315             DisplayMemberPath="
316                 NomComplet "
317             Width="300 "
318             HorizontalAlignment="Left
319                 "
320             Height="30 "
321             Margin="0,0,0,10"/>
322
323         <TextBlock FontStyle="Italic"
324             Foreground="#2E7D32">
325             <TextBlock.Text>
326                 <Binding Path="
327                     UtilisateurSelectionne.
328                     NomComplet "
329                     StringFormat="
330                         Auteur
331                         s lectionn :
332                         {0}"/>
333             </TextBlock.Text>
334         </TextBlock>
335
336     </StackPanel>
337 </Border>
338
339     <Button Content="Tester le mod le" Command
340         ="{Binding TestCommand}"
341         Height="45" Margin="0,5"
342         Background="#FF9800" Foreground="
343             White"
344         FontWeight="Bold" FontSize="14"/>
345
346     </StackPanel>
347 </GroupBox>
348
349 <!-- RESULTATS -->
350 <GroupBox Header="R sultats" Margin="0,15,0,0"
351     Visibility="{Binding ShowResults}">
352     <TextBox Text="{Binding ResultText}"
353         TextWrapping="Wrap" FontFamily="
354             Consolas"

```

```

339             MinHeight="200" MaxHeight="300"
340             VerticalScrollBarVisibility="Auto"
341             IsReadOnly="True" Padding="10"
                 Background="#FAFAFA"/>
342         </GroupBox>
343
344         <!-- HISTORIQUE -->
345         <GroupBox Header="Historique des exp riences"
346             Margin="0,15,0,15" Visibility="{Binding
                 ShowHistory}">
347             <ListBox ItemsSource="{Binding HistoriqueList}"
348                 MinHeight="150" MaxHeight="200"
349                 Margin="5" FontFamily="Consolas"/>
350         </GroupBox>
351
352     </StackPanel>
353 </ScrollViewer>
354
355 </Grid>
356 </Window>

```

Listing 21 – Fichier MainWindow.xaml

4.5.2 MainWindow.xaml.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6  using System.Windows;
7  using System.Windows.Controls;
8  using System.Windows.Data;
9  using System.Windows.Documents;
10 using System.Windows.Input;
11 using System.Windows.Media;
12 using System.Windows.Media.Imaging;
13 using System.Windows.Shapes;
14 using WPFClassificationGrainsDeBles.Services;
15
16 namespace WPFClassificationGrainsDeBles.View
17 {
18     public partial class MainWindow : Window
19     {
20         public MainWindow()
21         {
22             InitializeComponent();
23             DataContext = new ViewModels.MainWindowViewModel();
24         }
25

```

```

26     private void ShowAboutDialog(object sender, RoutedEventArgs
27         e)
28     {
29         MessageBox.Show(
30             "Classification de Grains de Bl \n" +
31             "Algorithme k-NN (k plus proches voisins)\n\n" +
32             "D evelopp e dans le cadre du cours P002\n" +
33             "Universit e du Qu ebec Rimouski (UQAR)\n\n" +
34             " 2026",
35             " propos",
36             MessageBoxButton.OK,
37             MessageBoxImage.Information);
38     }
39 }

```

Listing 22 – Fichier MainWindow.xaml.cs

4.6 ViewModels

Le répertoire `ViewModels` renferme les classes qui établissent le lien entre les modèles et les vues, en accord avec l'architecture MVVM.

4.6.1 MainWindowViewModel.cs

Le `MainWindowViewModel` sert de centre névralgique à l'application. Il prend en charge toute la logique sans connaître la façon dont l'interface est conçue.

Principales responsabilités : Il enregistre les informations suivantes : fichiers CSV téléchargés, valeur de k , sorte de distance, liste des utilisateurs API.

Il gère la présentation : affiche ou masque les pages (Accueil, Importation, Configuration, etc.).

Il réalise les tâches suivantes : l'importation des fichiers CSV, la formation du modèle, le test, et l'enregistrement dans la base de données.

Il répond aux clics : chaque bouton du menu déclenche une méthode ici (Importer, Tester, Quitter...)

Il interagit avec l'API : obtient les utilisateurs à partir de dummyjson.com.

Il conserve les résultats : consigne chaque essai dans la base de données SQL Server avec le nom de l'auteur.

Il conserve un historique : mémorise toutes les expériences effectuées.

```

1 using System;
2 using System.Collections.ObjectModel;
3 using System.ComponentModel;
4 using System.IO;
5 using System.Runtime.CompilerServices;
6 using System.Threading.Tasks;
7 using System.Windows;
8 using System.Windows.Input;
9 using Microsoft.Win32;
10 using WPFClassificationGrainsDeBles.Models;
11 using WPFClassificationGrainsDeBles.Services;

```

```

12
13 namespace WPFClassificationGrainsDeBles.ViewModels
14 {
15     public class MainWindowViewModel : INotifyPropertyChanged
16     {
17         private readonly SharedModel _sharedModel = new SharedModel
18             ();
19         private readonly UtilisateurService _utilisateurService =
20             new UtilisateurService();
21         private Utilisateur _utilisateurSelectionne;
22
23         public ObservableCollection<Utilisateur> Utilisateurs { get
24             ; set; } = new ObservableCollection<Utilisateur>();
25
26         public Utilisateur UtilisateurSelectionne
27         {
28             get => _utilisateurSelectionne;
29             set { _utilisateurSelectionne = value;
30                 OnPropertyChanged(); }
31         }
32
33         // Affichage de notre programme
34         private string _currentTitle = "Bienvenue sur l'application
35             de Classification des grains de bl - k-NN";
36         private Visibility _showWelcome = Visibility.Visible;
37         private Visibility _showImport = Visibility.Collapsed;
38         private Visibility _showKConfig = Visibility.Collapsed;
39         private Visibility _showDistanceConfig = Visibility.
40             Collapsed;
41         private Visibility _showTrain = Visibility.Collapsed;
42         private Visibility _showTest = Visibility.Collapsed;
43         private Visibility _showResults = Visibility.Collapsed;
44         private Visibility _showHistory = Visibility.Collapsed;
45         private string _importStatus = "Aucun fichier import ";
46         private string _resultText = "Les r sultats s'afficheront
47             dans cette section.";
48         private string _currentConfigText = "Aucune configuration";
49
50         // D clARATION des fichiers
51         private string _trainPath = "";
52         private string _testPath = "";
53
54         // Configuration distance avec k par default = 3
55         private int _kValue = 3;
56         private bool _isEuclidienne = true;
57         private bool _isManhattan = false;
58         private bool _kValidated = false;
59         private bool _distanceValidated = false;
60
61         // Historique
62         public ObservableCollection<string> HistoriqueList { get;

```

```

56         set; } = new ObservableCollection<string>();
57
58 // Propri t s
59 public string CurrentTitle
60 {
61     get => _currentTitle;
62     set { _currentTitle = value; OnPropertyChanged(); }
63 }
64
65 public Visibility ShowWelcome
66 {
67     get => _showWelcome;
68     set { _showWelcome = value; OnPropertyChanged(); }
69 }
70
71 public Visibility ShowImport
72 {
73     get => _showImport;
74     set { _showImport = value; OnPropertyChanged(); }
75 }
76
77 public Visibility ShowKConfig
78 {
79     get => _showKConfig;
80     set { _showKConfig = value; OnPropertyChanged(); }
81 }
82
83 public Visibility ShowDistanceConfig
84 {
85     get => _showDistanceConfig;
86     set { _showDistanceConfig = value; OnPropertyChanged(); }
87 }
88
89 public Visibility ShowTrain
90 {
91     get => _showTrain;
92     set { _showTrain = value; OnPropertyChanged(); }
93 }
94
95 public Visibility ShowTest
96 {
97     get => _showTest;
98     set { _showTest = value; OnPropertyChanged(); }
99 }
100
101 public Visibility ShowResults
102 {
103     get => _showResults;
104     set { _showResults = value; OnPropertyChanged(); }

```

```

105
106     public Visibility ShowHistory
107     {
108         get => _showHistory;
109         set { _showHistory = value; OnPropertyChanged(); }
110     }
111
112     public string ImportStatus
113     {
114         get => _importStatus;
115         set { _importStatus = value; OnPropertyChanged(); }
116     }
117
118     public string ResultText
119     {
120         get => _resultText;
121         set { _resultText = value; OnPropertyChanged(); }
122     }
123
124     public string CurrentConfigText
125     {
126         get => _currentConfigText;
127         set { _currentConfigText = value; OnPropertyChanged();
128             }
129     }
130
131     public string TrainPath
132     {
133         get => _trainPath;
134         set { _trainPath = value; OnPropertyChanged(); }
135     }
136
137     public string TestPath
138     {
139         get => _testPath;
140         set { _testPath = value; OnPropertyChanged(); }
141     }
142
143     public int KValue
144     {
145         get => _kValue;
146         set
147         {
148             if (value < 1) value = 1;
149             if (value > 30) value = 30;
150             _kValue = value;
151             OnPropertyChanged();
152         }
153     }
154
155     public bool IsEuclidienne

```

```

155 {
156     get => _isEuclidienne;
157     set
158     {
159         _isEuclidienne = value;
160         if (value) _isManhattan = false;
161         OnPropertyChanged();
162     }
163 }
164
165 public bool IsManhattan
166 {
167     get => _isManhattan;
168     set
169     {
170         _isManhattan = value;
171         if (value) _isEuclidienne = false;
172         OnPropertyChanged();
173     }
174 }
175
176 // Commandes
177 public ICommand BrowseTrainCommand { get; }
178 public ICommand BrowseTestCommand { get; }
179 public ICommand ImportCommand { get; }
180 public ICommand ValidateKAndContinueCommand { get; }
181 public ICommand ValidateDistanceAndContinueCommand { get; }
182 public ICommand TrainCommand { get; }
183 public ICommand TestCommand { get; }
184 public ICommand QuitCommand { get; }
185 public ICommand ChargerUtilisateursCommand { get; }
186
187 // Commandes de navigation
188 public ICommand NavigateHomeCommand { get; }
189 public ICommand NavigateImportCommand { get; }
190 public ICommand NavigateKConfigCommand { get; }
191 public ICommand NavigateDistanceConfigCommand { get; }
192 public ICommand NavigateResultsCommand { get; }
193 public ICommand NavigateHistoryCommand { get; }
194
195 // Constructeur
196 public MainWindowViewModel()
197 {
198     // Initialisation des commandes
199     BrowseTrainCommand = new RelayCommand(() => BrowseTrain
200         ());
201     BrowseTestCommand = new RelayCommand(() => BrowseTest()
202         );
203     ImportCommand = new RelayCommand(async () => await
204         ImportData());
205     ValidateKAndContinueCommand = new RelayCommand(() =>

```



```

        ValidateKAndContinue());
203 ValidateDistanceAndContinueCommand = new RelayCommand
        (() => ValidateDistanceAndContinue());
204 TrainCommand = new RelayCommand(() => Train());
205 TestCommand = new RelayCommand(() => Test());
206 QuitCommand = new RelayCommand(() => Application.
        Current.Shutdown());
207 ChargerUtilisateursCommand = new RelayCommand(async ()
        => await ChargerUtilisateurs());
208
209 // Commandes de navigation
210 NavigateHomeCommand = new RelayCommand(() => ShowHome()
        );
211 NavigateImportCommand = new RelayCommand(() =>
        ShowImportSection());
212 NavigateKConfigCommand = new RelayCommand(() =>
        ShowKConfigSection());
213 NavigateDistanceConfigCommand = new RelayCommand(() =>
        ShowDistanceConfigSection());
214 NavigateResultsCommand = new RelayCommand(() =>
        ShowResultsSection());
215 NavigateHistoryCommand = new RelayCommand(() =>
        ShowHistorySection());
216 }
217
218 // M thode chargement utilisateurs API
219 private async Task ChargerUtilisateurs()
220 {
221     try
222     {
223         var liste = await _utilisateurService.
            GetUtilisateursAsync();
224         Utilisateurs.Clear();
225         foreach (var u in liste)
226             Utilisateurs.Add(u);
227     }
228     catch (Exception ex)
229     {
230         MessageBox.Show($"Erreur chargement utilisateurs: {
            ex.Message}", "Erreur",
231             MessageBoxButton.OK, MessageBoxImage.Error);
232     }
233 }
234
235 // M thodes de navigation
236 private void ShowHome()
237 {
238     ShowWelcome = Visibility.Visible;
239     ShowImport = Visibility.Collapsed;
240     ShowKConfig = Visibility.Collapsed;
241     ShowDistanceConfig = Visibility.Collapsed;

```

```

242     ShowTrain = Visibility.Collapsed;
243     ShowTest = Visibility.Collapsed;
244     ShowResults = Visibility.Collapsed;
245     ShowHistory = Visibility.Collapsed;
246     CurrentTitle = "Bienvenue sur l'application de
        Classification des grains de bl - k-NN";
247 }
248
249 private void ShowImportSection()
250 {
251     ShowWelcome = Visibility.Collapsed;
252     ShowImport = Visibility.Visible;
253     ShowKConfig = Visibility.Collapsed;
254     ShowDistanceConfig = Visibility.Collapsed;
255     ShowTrain = Visibility.Collapsed;
256     ShowTest = Visibility.Collapsed;
257     ShowResults = Visibility.Collapsed;
258     ShowHistory = Visibility.Collapsed;
259     CurrentTitle = "1. Importer les donn es";
260 }
261
262 private void ShowKConfigSection()
263 {
264     ShowWelcome = Visibility.Collapsed;
265
266     if (!_sharedModel.IsDataLoaded)
267     {
268         MessageBox.Show("Veuillez d'abord importer les
            donn es.", "Information",
269             MessageBoxButton.OK, MessageBoxImage.
                Information);
270         ShowImportSection();
271         return;
272     }
273
274     ShowImport = Visibility.Collapsed;
275     ShowKConfig = Visibility.Visible;
276     ShowDistanceConfig = Visibility.Collapsed;
277     ShowTrain = Visibility.Collapsed;
278     ShowTest = Visibility.Collapsed;
279     ShowResults = Visibility.Collapsed;
280     ShowHistory = Visibility.Collapsed;
281     CurrentTitle = "2. Configurer la valeur de k";
282 }
283
284 private void ShowDistanceConfigSection()
285 {
286     ShowWelcome = Visibility.Collapsed;
287
288     if (!_kValidated)
289     {

```

```

290         MessageBox.Show("Veuillez d'abord valider la valeur
                                de k.", "Information",
291                             MessageBoxButtons.OK, MessageBoxIcon.
                                    Information);
292         ShowKConfigSection();
293         return;
294     }
295
296     ShowImport = Visibility.Collapsed;
297     ShowKConfig = Visibility.Collapsed;
298     ShowDistanceConfig = Visibility.Visible;
299     ShowTrain = Visibility.Collapsed;
300     ShowTest = Visibility.Collapsed;
301     ShowResults = Visibility.Collapsed;
302     ShowHistory = Visibility.Collapsed;
303     CurrentTitle = "3. Choisir la distance";
304 }
305
306 private void ShowTrainTestSections()
307 {
308     ShowWelcome = Visibility.Collapsed;
309     ShowImport = Visibility.Collapsed;
310     ShowKConfig = Visibility.Collapsed;
311     ShowDistanceConfig = Visibility.Collapsed;
312     ShowTrain = Visibility.Visible;
313     ShowTest = Visibility.Visible;
314     ShowResults = Visibility.Collapsed;
315     ShowHistory = Visibility.Collapsed;
316     CurrentTitle = "4. Entrer et 5. Tester le modèle";
317 }
318
319 private void ShowResultsSection()
320 {
321     ShowWelcome = Visibility.Collapsed;
322     ShowImport = Visibility.Collapsed;
323     ShowKConfig = Visibility.Collapsed;
324     ShowDistanceConfig = Visibility.Collapsed;
325     ShowTrain = Visibility.Collapsed;
326     ShowTest = Visibility.Collapsed;
327     ShowResults = Visibility.Visible;
328     ShowHistory = Visibility.Collapsed;
329     CurrentTitle = "Résultats de classification";
330 }
331
332 private void ShowHistorySection()
333 {
334     ShowWelcome = Visibility.Collapsed;
335     ShowImport = Visibility.Collapsed;
336     ShowKConfig = Visibility.Collapsed;
337     ShowDistanceConfig = Visibility.Collapsed;
338     ShowTrain = Visibility.Collapsed;

```

```

339         ShowTest = Visibility.Collapsed;
340         ShowResults = Visibility.Collapsed;
341         ShowHistory = Visibility.Visible;
342         CurrentTitle = "Historique des exp riences";
343     }
344
345     private void BrowseTrain()
346     {
347         var dialog = new OpenFileDialog
348         {
349             Title = "S lectionner le fichier train.csv",
350             Filter = "CSV files (*.csv)|*.csv|All files (*.*)|*.*"
351         };
352         if (dialog.ShowDialog() == true)
353             TrainPath = dialog.FileName;
354     }
355
356     private void BrowseTest()
357     {
358         var dialog = new OpenFileDialog
359         {
360             Title = "S lectionner le fichier test.csv",
361             Filter = "CSV files (*.csv)|*.csv|All files (*.*)|*.*"
362         };
363         if (dialog.ShowDialog() == true)
364             TestPath = dialog.FileName;
365     }
366
367     private async System.Threading.Tasks.Task ImportData()
368     {
369         if (string.IsNullOrEmpty(TrainPath) || string.
370             IsNullOrEmpty(TestPath))
371         {
372             ImportStatus = "Veuillez s lectionner les deux
373                 fichiers!";
374             return;
375         }
376
377         try
378         {
379             var trainGrains = DataConverter.ConversionListe(
380                 TrainPath);
381             _sharedModel.TrainingData = new EnsembleDonnees();
382             DataConverter.SaveEchantillon(trainGrains,
383                 _sharedModel.TrainingData);
384
385             var testGrains = DataConverter.ConversionListe(
386                 TestPath);
387             _sharedModel.TestData = new EnsembleDonnees();

```

```

383         DataConverter.SaveEchantillon(testGrains,
384             _sharedModel.TestData);
385
386         _sharedModel.IsDataLoaded = true;
387         _sharedModel.InitializeClassifier();
388
389         ImportStatus = $"Succ s: {_sharedModel.
390             TrainingData.Taille()} train, {_sharedModel.
391             TestData.Taille()} test";
392
393         _kValidated = false;
394         _distanceValidated = false;
395
396         ShowKConfigSection();
397     }
398     catch (Exception ex)
399     {
400         ImportStatus = $"Erreur: {ex.Message}";
401         _sharedModel.IsDataLoaded = false;
402     }
403 }
404
405 private void ValidateKAndContinue()
406 {
407     if (!_sharedModel.IsDataLoaded)
408     {
409         MessageBox.Show("Veuillez d'abord importer les
410             donn es!", "Erreur",
411             MessageBoxButton.OK, MessageBoxImage.Error);
412         ShowImportSection();
413         return;
414     }
415
416     _sharedModel.K = KValue;
417     _kValidated = true;
418     ResultText = $"k = {KValue} valid . Choisissez
419         maintenant la distance.";
420     ShowDistanceConfigSection();
421 }
422
423 private void ValidateDistanceAndContinue()
424 {
425     if (!_sharedModel.IsDataLoaded)
426     {
427         MessageBox.Show("Veuillez d'abord importer les
428             donn es!", "Erreur",
429             MessageBoxButton.OK, MessageBoxImage.Error);
430         ShowImportSection();
431         return;
432     }
433 }

```

```

428         if (!_kValidated)
429         {
430             MessageBox.Show("Veuillez d'abord valider la valeur
                                de k!", "Erreur",
431                               MessageBoxButtons.OK, MessageBoxIcon.Error);
432             ShowKConfigSection();
433             return;
434         }
435
436         string distance = IsEuclidienne ? "Euclidienne" : "
                                Manhattan";
437         _sharedModel.SetDistance(distance);
438         _distanceValidated = true;
439
440         UpdateCurrentConfigText();
441         ShowTrainTestSections();
442         ResultText = $"Distance {distance} valide. Vous
                                pouvez maintenant entraîner et tester.";
443     }
444
445     private void UpdateCurrentConfigText()
446     {
447         string distance = IsEuclidienne ? "Euclidienne" : "
                                Manhattan";
448         CurrentConfigText = $"Configuration actuelle :\n" +
449                               $"    - k = {_sharedModel.K}\n" +
450                               $"    - Distance = {distance}\n" +
451                               $"    - Données d'entraînement = {
                                    _sharedModel.TrainingData?.Taille
452                                     () ?? 0}    chantillons \n" +
453                               $"    - Données de test = {
                                    _sharedModel.TestData?.Taille()
454                                     ?? 0}    chantillons ";
455     }
456
457     private void Train()
458     {
459         if (!_sharedModel.IsDataLoaded)
460         {
461             MessageBox.Show("Veuillez d'abord importer les
                                données!", "Erreur",
462                               MessageBoxButtons.OK, MessageBoxIcon.Error);
463             ShowImportSection();
464             return;
465         }
466
467         if (!_kValidated || !_distanceValidated)
468         {
469             MessageBox.Show("Veuillez d'abord configurer k et
                                la distance!", "Erreur",
470                               MessageBoxButtons.OK, MessageBoxIcon.Error);

```

```

469         ShowKConfigSection();
470         return;
471     }
472
473     _sharedModel.InitializeClassifier();
474     ResultText = $"Mod le entra n avec succ s\n\n" +
475         $"    - k = {_sharedModel.K}\n" +
476         $"    - Distance = {_sharedModel.
477             GetDistanceName()}\n" +
478         $"    - Donn es d'entra nement = {
479             _sharedModel.TrainingData.Taille()
480             chantillons \n\n" +
481         $"Vous pouvez maintenant tester le mod le.
482         ";
483
484     MessageBox.Show("Entra nement termin avec succ s.",
485         "Succ s",
486         MessageBoxButton.OK, MessageBoxImage.Information);
487 }
488
489 private void Test()
490 {
491     if (!_sharedModel.IsDataLoaded)
492     {
493         MessageBox.Show("Veuillez d'abord importer les
494             donn es.", "Erreur",
495             MessageBoxButton.OK, MessageBoxImage.Error);
496         ShowImportSection();
497         return;
498     }
499
500     if (_sharedModel.Classifier == null)
501     {
502         MessageBox.Show("Veuillez d'abord entra ner le
503             mod le.", "Erreur",
504             MessageBoxButton.OK, MessageBoxImage.Error);
505         return;
506     }
507
508     if (UtilisateurSelectionne == null)
509     {
510         MessageBox.Show("Veuillez s lectionner un auteur
511             avant de tester.", "Erreur",
512             MessageBoxButton.OK, MessageBoxImage.Error);
513         return;
514     }
515
516     try
517     {
518         var evaluation = new EvaluationPerformance();
519         evaluation.Evaluer(_sharedModel.K, _sharedModel.

```

```

512         DistanceStrategy,
513             _sharedModel.TrainingData,
514             _sharedModel.TestData);
515
516     double accuracy = evaluation.CalculerAccuracy() *
517         100;
518     string distanceName = _sharedModel.GetDistanceName
519         ();
520
521     SauvegarderDonnee(accuracy, distanceName);
522
523     string resultat = $"R SULTAT DU TEST\n" +
524         $"
525         \n" +
526         $"Param tres du classifieur :\n"
527         +
528         $"          k = {_sharedModel.K}\n" +
529         $"          Distance = {distanceName
530             }\n" +
531         $"          Donn es test = {
532             _sharedModel.TestData.Taille()}
533             chantillons \n\n" +
534         $"Pr cision (Accuracy) = {
535             accuracy:F2}%\n" +
536         $"Auteur = {UtilisateurSelectionne
537             .NomComplet}\n" +
538         $"Donn es sauvegard es dans la
539             base\n" +
540         $"
541
542         ";
543
544     ResultText = resultat;
545
546     string dateStr = DateTime.Now.ToString("yyyy-MM-dd
547         HH:mm:ss");
548     string historiqueEntry = $"{{dateStr}} | k={
549         _sharedModel.K} | {distanceName} | Accuracy={
550         accuracy:F2}% | Auteur={UtilisateurSelectionne.
551         NomComplet}";
552     HistoriqueList.Insert(0, historiqueEntry);
553
554     string jsonPath = Path.Combine(AppDomain.
555         CurrentDomain.BaseDirectory, "historique.json");
556     evaluation.SauvegarderJsonGlobal(jsonPath,
557         _sharedModel.K, _sharedModel.DistanceStrategy,
558         _sharedModel.
559             TrainingData,
560             _sharedModel.
561             TestData);

```



```

539         ShowResultsSection();
540
541         MessageBox.Show($"Test termin ! Pr cision: {
542             accuracy:F2}% ", "R sultats ",
543             MessageBoxButton.OK, MessageBoxImage.
544                 Information);
545     }
546     catch (Exception ex)
547     {
548         ResultText = $"Erreur lors du test: {ex.Message}";
549         MessageBox.Show($"Erreur lors du test: {ex.Message}
550             ", "Erreur ",
551             MessageBoxButton.OK, MessageBoxImage.Error);
552     }
553 }
554
555 private void SauvegarderDonnee(double accuracy, string
556     distanceName)
557 {
558     using (var context = new
559         ClassificationGrainDeBlesContext())
560     {
561         var donnee = new Models.Donnee
562         {
563             k = _sharedModel.K,
564             Distance = accuracy,
565             donnee_Tester = Path.GetFileName(TestPath),
566             precision = $"{{accuracy:F2}}%",
567             AuteurNom = UtilisateurSelectionne?.NomComple
568                 ?? "Anonyme"
569         };
570
571         context.donnees.Add(donnee);
572         context.SaveChanges();
573     }
574 }
575
576 // INotifyPropertyChanged
577 public event PropertyChangedEventHandler PropertyChanged;
578
579 protected void OnPropertyChanged([CallerMemberName] string
580     propertyName = null)
581 {
582     PropertyChanged?.Invoke(this, new
583         PropertyChangedEventArgs(propertyName));
584 }
585 }
586 }

```

Listing 23 – Fichier MainWindowViewModel.cs

4.6.2 RelayCommand.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6 using System.Windows.Input;
7
8 namespace WPFClassificationGrainsDeBles.ViewModels
9 {
10     public class RelayCommand : ICommand
11     {
12         private readonly Action<object> _executeWithParam;
13         private readonly Action _executeWithoutParam;
14         private readonly Func<object, bool> _canExecute;
15
16         // Constructeur avec param tre object
17         public RelayCommand(Action<object> execute, Func<object,
18             bool> canExecute = null)
19         {
20             _executeWithParam = execute ?? throw new
21                 ArgumentNullException(nameof(execute));
22             _canExecute = canExecute;
23         }
24
25         // Constructeur sans param tre
26         public RelayCommand(Action execute, Func<bool> canExecute =
27             null)
28         {
29             _executeWithoutParam = execute ?? throw new
30                 ArgumentNullException(nameof(execute));
31             if (canExecute != null)
32                 _canExecute = (p) => canExecute();
33         }
34
35         public event EventHandler CanExecuteChanged
36         {
37             add { CommandManager.RequerySuggested += value; }
38             remove { CommandManager.RequerySuggested -= value; }
39         }
40
41         public bool CanExecute(object parameter)
42         {
43             return _canExecute == null || _canExecute(parameter);
44         }
45
46         public void Execute(object parameter)
47         {
48             if (_executeWithParam != null)
49                 _executeWithParam(parameter);
50         }
51     }
52 }
```

```

46         else if (_executeWithoutParam != null)
47             _executeWithoutParam();
48     }
49 }
50 }

```

Listing 24 – Classe RelayCommand.cs

4.7 Classe Converters.cs

La classe `Converters` fournit des convertisseurs de valeurs personnalisés pour le databinding dans l'interface WPF.

```

1  using System;
2  using System.Collections.Generic;
3  using System.Globalization;
4  using System.Linq;
5  using System.Text;
6  using System.Threading.Tasks;
7  using System.Windows;
8  using System.Windows.Data;
9
10 namespace WPFClassificationGrainsDeBles
11 {
12     public class BoolToVisibilityConverter : IValueConverter
13     {
14         public object Convert(object value, Type targetType, object
            parameter, CultureInfo culture)
15         {
16             if (value is bool boolValue)
17                 return boolValue ? Visibility.Visible : Visibility.
                    Collapsed;
18             return Visibility.Collapsed;
19         }
20
21         public object ConvertBack(object value, Type targetType,
            object parameter, CultureInfo culture)
22         {
23             throw new NotImplementedException();
24         }
25     }
26
27     public class ZeroToVisibilityConverter : IValueConverter
28     {
29         public object Convert(object value, Type targetType, object
            parameter, CultureInfo culture)
30         {
31             if (value is int count)
32                 return count == 0 ? Visibility.Visible : Visibility.
                    Collapsed;
33             return Visibility.Visible;
34         }

```

```

35
36     public object ConvertBack(object value, Type targetType,
37         object parameter, CultureInfo culture)
38     {
39         throw new NotImplementedException();
40     }
41
42     public class StringToBoolConverter : IValueConverter
43     {
44         public object Convert(object value, Type targetType, object
45             parameter, CultureInfo culture)
46         {
47             return value?.ToString() == parameter?.ToString();
48         }
49
50         public object ConvertBack(object value, Type targetType,
51             object parameter, CultureInfo culture)
52         {
53             return (bool)value ? parameter?.ToString() : null;
54         }
55     }
56 }

```

Listing 25 – Classe Converters.cs

4.8 App.xaml.cs

Ce code représente la classe de démarrage pour l'application WPF. Elle constitue l'entrée principale de l'application.

```

1  using System;
2  using System.Collections.Generic;
3  using System.Configuration;
4  using System.Data;
5  using System.Linq;
6  using System.Threading.Tasks;
7  using System.Windows;
8
9  namespace WPFClassificationGrainsDeBles
10 {
11     public partial class App : Application
12     {
13         protected override void OnStartup(StartupEventArgs e)
14         {
15             base.OnStartup(e);
16         }
17     }
18 }

```

Listing 26 – Fichier App.xaml.cs

5 Conclusion

Ce projet a permis de développer une application complète de classification de grains de blé avec une interface graphique WPF respectant l'architecture MVVM, la persistance des données via Entity Framework Core et l'intégration d'une API REST publique.

Références

- [1] Microsoft. (2025). *Documentation C# : Guide du développeur*. Microsoft Learn. Consulté le 10 mai 2026. <https://learn.microsoft.com/fr-fr/dotnet/csharp/tour-of-csharp/overview>
- [2] Microsoft. (2025). *Windows Presentation Foundation (WPF) : Présentation*. Microsoft Learn. Consulté le 10 mai 2026. <https://learn.microsoft.com/fr-fr/dotnet/desktop/wpf/>
- [3] Microsoft. (2025). *Le modèle MVVM (Model-View-ViewModel) pour applications WPF*. Microsoft Learn. Consulté le 10 mai 2026. <https://learn.microsoft.com/fr-fr/dotnet/architecture/maui/> <https://learn.microsoft.com/fr-fr/dotnet/architecture/maui/mvvm>
- [4] Microsoft. (2025). *Entity Framework Core : Guide de démarrage*. Microsoft Learn. Consulté le 10 mai 2026. <https://learn.microsoft.com/fr-fr/ef/core/>
- [5] dummyjson. (2025). *DummyJSON API Documentation - Fake API for testing*. Consulté le 10 mai 2026. <https://dummyjson.com/docs/users>
- [6] INF11207 – Programmation orientée objet II – Prof. Yacine Yaddaden, Ph. D.